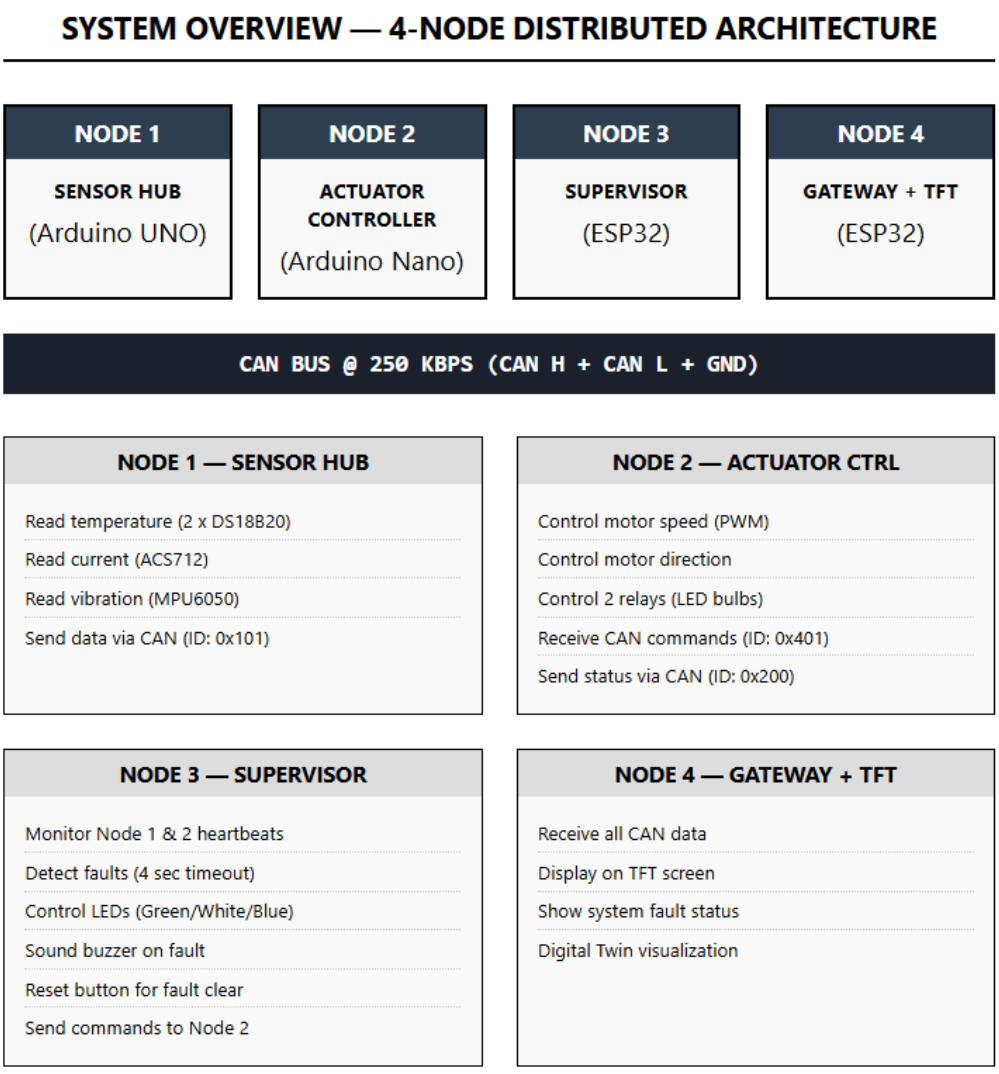
CHAPTER 1

INTRODUCTION

1.1 Overview

In modern industrial automation, reliability is paramount. Systems must continue operating even when individual components fail. This project addresses this need by implementing a **Fault-Tolerant Distributed Embedded Control System** using CAN (Controller Area Network) protocol.

CAN bus, originally developed by Bosch in 1986 for automotive applications, has become the industry standard for distributed control systems. It allows multiple microcontrollers (nodes) to communicate with each other using just two wires (CAN High and CAN Low), eliminating the need for complex wiring harnesses.



All nodes communicate over CAN bus. Node 3 acts as fault supervisor. Node 4 provides visual interface.

1.2 Problem Statement

In traditional centralized control systems, a single controller failure can bring down the entire system. This is unacceptable in critical applications such as:

- **Industrial manufacturing lines** - A single controller failure can halt production
- **Automotive systems** - Brake or engine controller failure can cause accidents
- **Medical equipment** - Patient monitoring system failure can be life-threatening
- **Building management** - HVAC or fire alarm failure can cause property damage

Key Problems Addressed:

Problem	Traditional Approach	Our Solution
Single point of failure	One master controller	Distributed nodes with no single master
No fault detection	Manual inspection	Automatic heartbeat monitoring
No recovery mechanism	Manual reset	Reset button for quick recovery
No real-time status	Separate monitoring system	TFT display showing all node status
Complex wiring	Multiple wires between controllers	CAN bus with just 2 wires + ground

1.3 Objective of the Project

Primary Objectives

#	Objective
1	Design a 4-node distributed system using CAN bus
2	Implement real-time sensor monitoring (temperature, current, vibration)
3	Implement actuator control (motor speed, direction, relays)
4	Implement fault detection using heartbeat monitoring

#	Objective
5	Provide visual and audible fault indication
6	Display all system data on TFT screen
7	Provide manual fault recovery via reset button

Secondary Objectives

- Demonstrate CAN bus communication between different microcontroller platforms (UNO, Nano, ESP32)
- Implement redundant temperature sensing (primary + backup DS18B20)
- Simulate thermal load using LED bulbs controlled by relays
- Achieve 250KBPS CAN communication speed
- Detect node failures within 4 seconds

1.4 Literature Survey

1.4.1 CAN Bus Protocol

The Controller Area Network (CAN) bus was developed by **Bosch** in 1986 and standardized as ISO 11898. Key features include:

Feature	Specification
Multi-master	Any node can initiate communication
Message-based	Messages have IDs, not node addresses
Arbitration	Lower ID has higher priority
Error detection	CRC, ACK, bit monitoring, frame check
Data rate	Up to 1 MBPS
Bus length	Up to 40m at 1 MBPS, 500m at 125 KBPS
Nodes	Up to 110 nodes per bus

CAN Frame Structure (Standard 11-bit ID):

Field	Bits	Description
SOF	1	Start of Frame (dominant 0)
ID	11	Message identifier (lower = higher priority)
RTR	1	Remote Transmission Request
Control	6	IDE bit + DLC (data length)
Data	0-64	Actual data bytes (0-8 bytes typical)
CRC	15	Cyclic Redundancy Check
ACK	2	Acknowledgment slot
EOF	7	End of Frame

1.4.2 Related Work

Author/Year	Work	Limitations
Bosch (1986)	CAN Protocol Specification	Hardware only, no software implementation
Arduino.cc (2010) / MCP2515 Library	CAN Library for Arduino	Single node only, no multi-node testing
Seeed Studio (2015)	CAN Bus Shield	Only for Arduino, no ESP32 support
Bodmer (2019)	TFT_eSPI Library	Display library only, no CAN integration

Gap in Literature:

- No complete implementation of 4-node fault-tolerant system
- No integration of CAN with TFT display for real-time status
- No multi-platform (UNO, Nano, ESP32) CAN network documented

Our Contribution:

- Complete 4-node fault-tolerant system

- CAN communication between different microcontroller platforms
- Real-time TFT display integrated with CAN
- Heartbeat monitoring with 4-second fault detection

1.5 Existing and Proposed System

1.5.1 Existing System

Traditional industrial control systems use:

Component	Existing Approach
Architecture	Centralized (single PLC)
Communication	4-20mA current loops or RS-485
Fault detection	Manual inspection
Status display	Separate HMI panel
Wiring	Each sensor/actuator wired separately

Disadvantages:

- Single point of failure (PLC failure stops everything)
- Complex wiring (each device needs separate wires)
- No automatic fault detection
- Expensive HMI panels
- Difficult to expand or modify

1.5.2 Proposed System

Component	Proposed Approach
Architecture	Distributed (4 nodes, no single master)
Communication	CAN bus (2 wires + ground)
Fault detection	Automatic heartbeat monitoring
Status display	TFT display integrated with CAN
Wiring	All nodes share same 2 wires

Advantages of Proposed System:

- No single point of failure
- Minimal wiring (all nodes share CAN bus)
- Automatic fault detection
- Low-cost TFT display
- Easy to add more nodes

1.6 Hardware and Software Components Used

1.6.1 Hardware Components Summary

S.No	Component	Quantity	Purpose
1	Arduino UNO R3	1	Node 1 - Sensor Hub
2	Arduino Nano	1	Node 2 - Actuator Controller
3	ESP32 Dev Board	2	Node 3 & Node 4
4	MCP2515 CAN Module	2	CAN for Node 1 & 2
5	TJA1050 CAN Transceiver	2	CAN for Node 3 & 4
6	MPU6050	1	Vibration sensing
7	ACS712 (5A)	1	Current sensing
8	DS18B20	2	Temperature sensing (primary + backup)
9	L298N Motor Driver	1	Motor control
10	DC Geared Motor (300 RPM)	1	Mechanical load
11	5V 2-Channel Relay	1	LED bulb control
12	12V LED Bulb	2	Thermal load simulation
13	2.8" SPI TFT Display	1	Visualization
14	LEDs (Green, White, Blue)	3	Status indicators
15	Active Buzzer (5V)	1	Audible alarm
16	Push Button	1	Reset
17	12V 2A Adapter	1	Main power

S.No	Component	Quantity	Purpose
18	LM2596 Buck Converter	1	12V → 5V
19	120Ω Resistors	2	CAN termination
20	4.7kΩ Resistors	3	DS18B20 pull-ups
21	220Ω Resistors	3	LED current limiting
22	Breadboards	4	Prototyping
23	Jumper Wires	Set	Connections

1.6.2 Software Components

Software / Library	Version	Purpose
Arduino IDE	2.0+	Code development and upload
CAN.h Library	1.0	CAN communication
TFT_eSPI Library	2.5+	TFT display driver
MPU6050 Library	1.1.1	Accelerometer interface
OneWire Library	2.3.7	DS18B20 protocol
DallasTemperature	3.9.0	Temperature sensor
mcp2515.h	1.0	MCP2515 CAN controller

1.7 Organization of the Project

Chapter	Title	Description
1	**Introduction**	Overview, objectives, literature survey
2	**System Design**	Block diagram, working mechanism, CAN protocol

Chapter	Title	Description
3	**Hardware Design**	Component specifications, pinouts
4	**Hardware Implementation**	Circuit diagram, wiring
5	**Software Description**	Libraries, functions, APIs
6	**Software Implementation**	Flowcharts, algorithms
7	**Experimental Results**	Test results, outputs, photos
8	**Applications & Analysis**	Applications, advantages, disadvantages
9	**Conclusion & Future Scope**	Summary, future enhancements

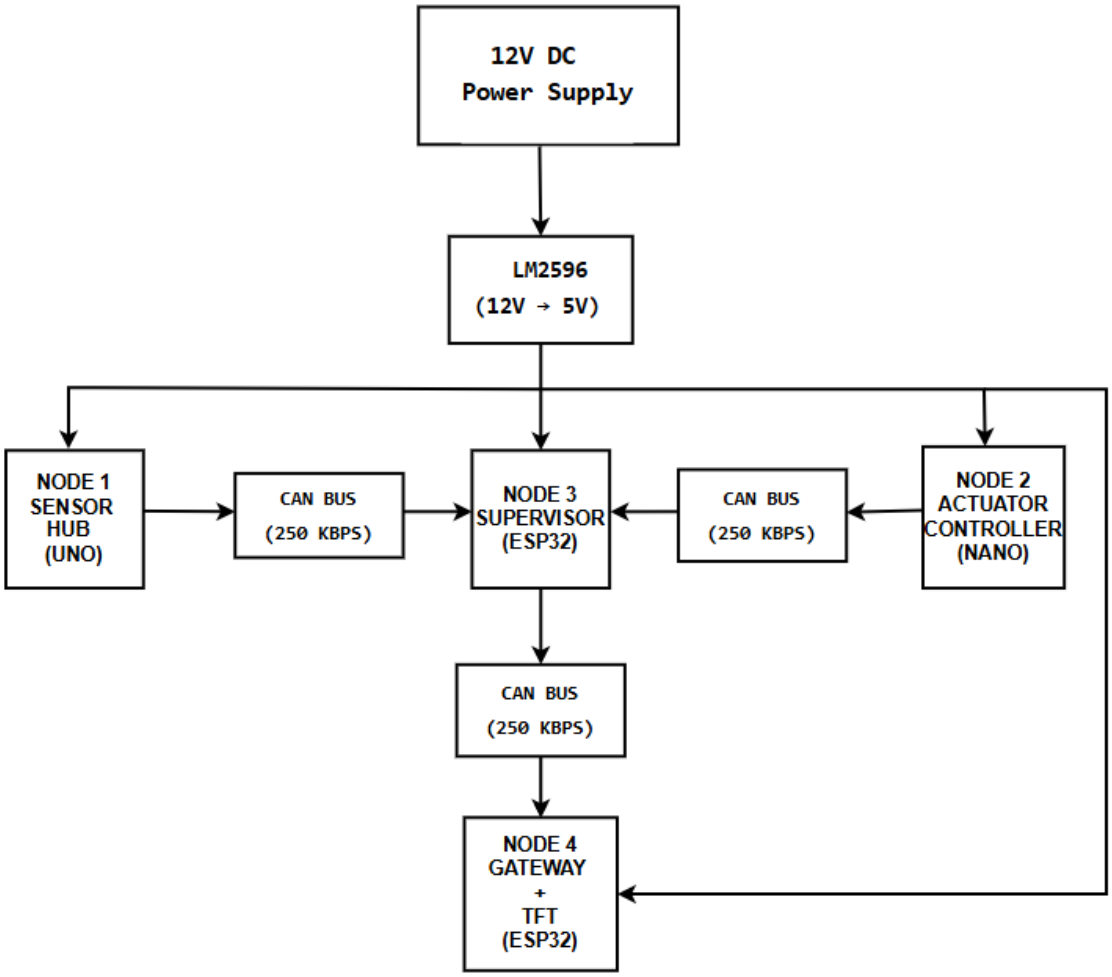
1.8 Summary

This chapter introduced the Fault-Tolerant Distributed Embedded Control System, its need in industrial applications, and the objectives of the project. A literature survey covered CAN bus protocol and related work. The proposed system architecture was compared with existing centralized systems. A complete list of hardware and software components was provided.

CHAPTER 2

SYSTEM DESIGN AND OPERATION

2.1 Block Diagram



2.2 Working Mechanism

2.2.1 CAN Communication Protocol

The CAN bus uses a **multi-master** architecture where any node can initiate communication. Messages are broadcast to all nodes, and each node decides whether to accept or ignore based on the message ID.

CAN Frame Transmission:

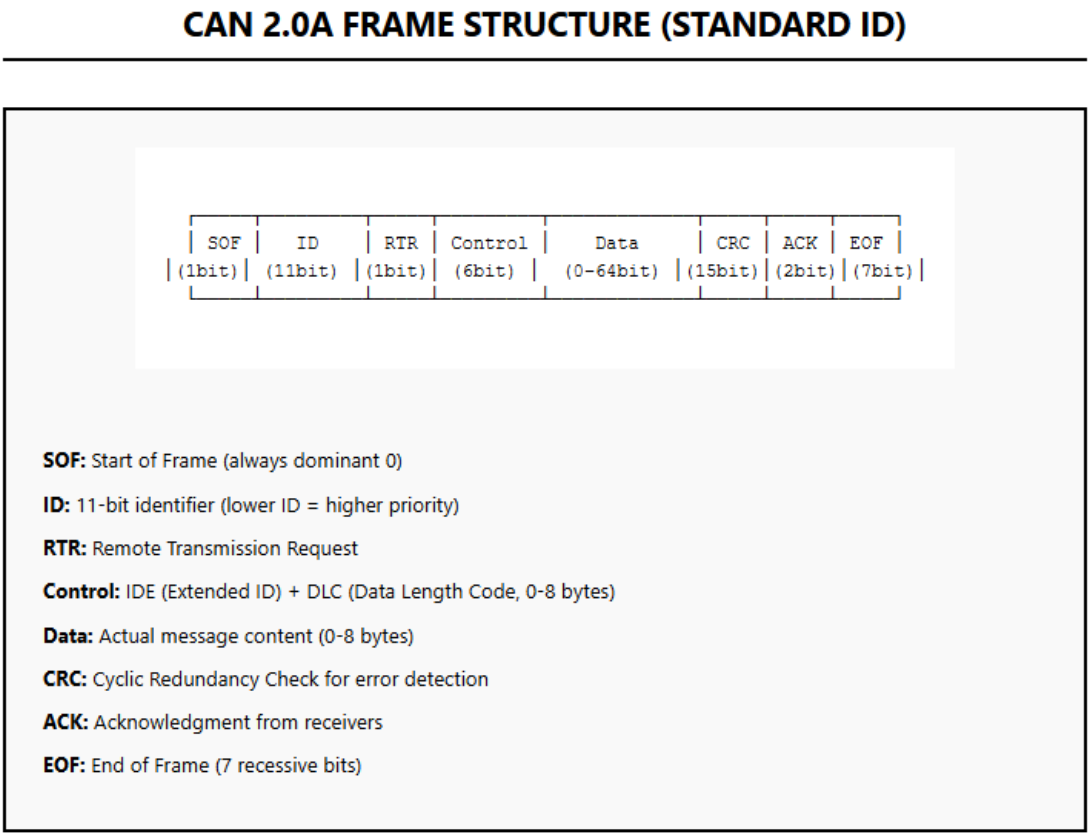


Figure 2.2: CAN Message Frame Structure

2.2.2 Node 1 - Sensor Hub Operation

Step-by-step operation:

1. **Initialize** all sensors (MPU6050, ACS712, 2xDS18B20) and CAN module
2. **Read sensors** every 500ms:
    - Request temperature from both DS18B20 sensors
    - Read analog value from ACS712, convert to current (A)
    - Read acceleration data from MPU6050 (X, Y, Z axes)
3. **Send CAN message** every 2 seconds:
    - CAN ID: 0x101
    - Data bytes: [Node ID, Message#, Time, Temp1, Temp2, Current×100, AccelX high, AccelX low]
4. **Repeat** indefinitely

2.2.3 Node 2 - Actuator Controller Operation

Step-by-step operation:

1. **Initialize** motor driver, relays, and CAN module
2. **Send status** every 2 seconds:
    - Current motor speed
    - Motor direction (Forward/Reverse)
    - Relay 1 state (ON/OFF)
    - Relay 2 state (ON/OFF)
    - Emergency stop status
3. **Check for CAN commands** continuously:
    - If command received (CAN ID: 0x401), parse target, command, value
    - Execute command if target is Node 2 or broadcast (0)
4. **Update motor output** based on current settings
5. **Repeat**

2.2.4 Node 3 - Supervisor Operation

Step-by-step operation:

1. **Initialize** LEDs, buzzer, button, and CAN module
2. **Send heartbeat** every 2 seconds (CAN ID: 0x303)
3. **Monitor Node 1 and Node 2**:
    - Track last data received time
    - If no data for 4 seconds, declare fault
4. **Send control commands** to Node 2 every 10 seconds:
    - Cycle through: Speed 150 → Forward → Relay1 ON → Relay2 ON → Relay1 OFF → Relay2 OFF → Speed 100 → Reverse → Stop
5. **Update LEDs**:
    - Green LED ON if Node 1 active
    - White LED ON if Node 2 active
    - Blue LED blinking if fault detected
6. **Sound buzzer** if fault detected
7. **Check reset button** - clear all faults when pressed

2.2.5 Node 4 - Gateway & Display Operation

Step-by-step operation:

1. **Initialize** TFT display and CAN module
2. **Receive CAN messages** from Node 1, Node 2, Node 3
3. **Track node status**:
    - Last data received time for each node
    - If no data for 4 seconds, mark node as fault
4. **Update TFT display** every 200ms:
    - Show Node 1 sensor data (temperatures, current, acceleration)

    - Show Node 2 actuator data (motor speed, direction, relays)
    - Show system status (NORMAL/FAULT/WAITING)
5. **Send heartbeat** every 2 seconds (CAN ID: 0x304)
6. **Repeat**

2.2.6 Fault Detection Mechanism

Condition	Detection Method	Timeout	Action
Node 1 fails	No sensor data (ID: 0x101)	4 seconds	Blue LED blinks, buzzer sounds, TFT shows SYSTEM FAULT
Node 2 fails	No status data (ID: 0x200)	4 seconds	Blue LED blinks, buzzer sounds, TFT shows SYSTEM FAULT
Node 3 fails	No heartbeat (ID: 0x303)	4 seconds	TFT shows SYSTEM FAULT
All nodes active	Heartbeats received	-	Green/White LEDs ON, TFT shows SYSTEM NORMAL

CHAPTER 3

HARDWARE DESIGN AND DESCRIPTION

3.1 Microcontrollers

3.1.1 Arduino UNO R3 (Node 1)

Specification	Value
Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Digital I/O Pins	14 (6 PWM)
Analog Input Pins	6
Flash Memory	32 KB
SRAM	2 KB
Clock Speed	16 MHz

3.1.2 Arduino Nano (Node 2)

Specification	Value
Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Digital I/O Pins	14 (6 PWM)
Analog Input Pins	6

Specification	Value
Flash Memory	32 KB
SRAM	2 KB
Clock Speed	16 MHz

3.1.3 ESP32 Dev Board (Node 3 & Node 4)

Specification	Value
Microcontroller	ESP32 (Tensilica Xtensa LX6)
Operating Voltage	3.3V
Input Voltage	5V (via USB) or 5V pin
Digital I/O Pins	25
Analog Input Pins	15
Flash Memory	4 MB
SRAM	520 KB
Clock Speed	240 MHz
Wi-Fi	802.11 b/g/n
Bluetooth	v4.2 BR/EDR and BLE

3.2 CAN Communication Modules

3.2.1 MCP2515 CAN Module (For Node 1 & 2)

Specification	Value
Controller	MCP2515

Specification	Value
Transceiver	TJA1050
CAN Specification	2.0B (1 MBPS)
Interface	SPI (10 MHz)
Operating Voltage	5V
Crystal Frequency	8 MHz
Transmit Buffers	3
Receive Buffers	2
Filters	6 (29-bit)
Masks	2 (29-bit)

3.2.2 TJA1050 CAN Transceiver (For Node 3 & 4)

Specification	Value
Transceiver	TJA1050
Data Rate	Up to 1 MBPS
Operating Voltage	4.75-5.25V
Nodes	Up to 110
Standby Current	Low (typical)
ESD Protection	High

3.3 Sensors

3.3.1 MPU6050 Accelerometer/Gyroscope (Node 1)

Specification	Value
Accelerometer Range	±2g, ±4g, ±8g, ±16g
Gyroscope Range	±250, ±500, ±1000, ±2000 °/s
Interface	I2C (SDA, SCL)
Operating Voltage	3.3V - 5V
Current Consumption	3.9 mA (typical)
I2C Address	0x68 (default) or 0x69 (if AD0 high)

3.3.2 ACS712 Current Sensor (Node 1)

Specification	Value
Current Range	±5A
Sensitivity	185 mV/A
Output Voltage	2.5V at 0A
Operating Voltage	5V
Bandwidth	80 kHz

Formula: Current (A) = (Vout - 2.5) / 0.185

3.3.3 DS18B20 Temperature Sensor (Node 1)

Specification	Value
Temperature Range	-55°C to +125°C

Specification	Value
Accuracy	±0.5°C (-10°C to +85°C)
Resolution	9-12 bit (programmable)
Interface	1-Wire
Operating Voltage	3V - 5.5V

Pull-up resistor: 4.7kΩ required between VDD and DATA pin

3.4 Actuators

3.4.1 L298N Motor Driver (Node 2)

Specification	Value
Supply Voltage	5V - 35V (logic), 12V (motor)
Output Current	2A per channel (peak 3A)
Logic Voltage	5V
PWM Frequency	Up to 40 kHz
Channels	2 (H-bridge)

3.4.2 5V 2-Channel Relay Module (Node 2)

Specification	Value
Trigger Type	Low Level (LOW = ON)
Operating Voltage	5V
Current	15-20 mA per channel

Specification	Value
Load	250V 10A AC or 30V 10A DC
Optocoupler Isolation	Yes

Pinout:

- DC+ → 5V
- DC- → GND
- CH1 → D5 (Relay 1 control)
- CH2 → D6 (Relay 2 control)
- COM → 12V (+)
- NO → LED Bulb (+)
- LED Bulb (-) → GND

3.5 Display Module

3.5.1 2.8" SPI TFT Display (Node 4)

Specification	Value
Driver	ILI9341 (SPI)
Resolution	240 × 320 pixels
Color Depth	65K / 262K colors
Interface	SPI (4-wire)
Operating Voltage	3.3V
Touch Screen	Resistive (optional)

Pinout:

- VCC → 3.3V
- GND → GND
- CS → GPIO15
- DC → GPIO2
- RST → GPIO4
- MOSI → GPIO23

- SCK → GPIO18
- LED → 3.3V

3.6 Indicators

3.6.1 LEDs (Node 3)

LED	Color	ESP32 Pin	Purpose
LED1	**Green**	GPIO2	Node 1 Active
LED2	**White**	GPIO4	Node 2 Active
LED3	**Blue**	GPIO15	Fault Indicator

Resistor: 220Ω in series with each LED

3.6.2 Active Buzzer (Node 3)

Specification	Value
Operating Voltage	5V
Sound Output	85-95 dB
Frequency	2300-2500 Hz

Pin: GPIO16 (LOW = OFF, HIGH = ON)

3.6.3 Push Button (Node 3)

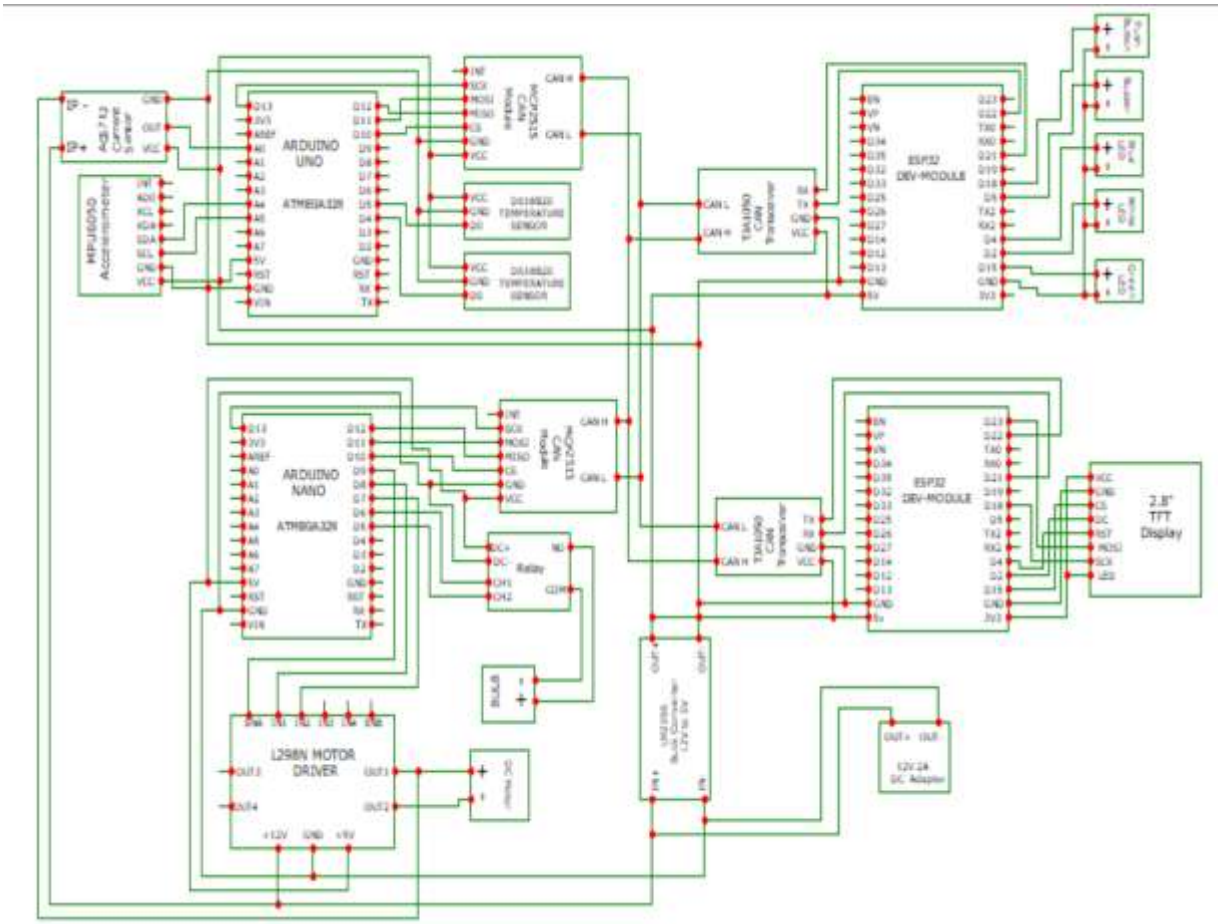
Specification	Value
Type	Tactile Switch
Connection	GPIO17 to GND
Internal Pull-up	Enabled (INPUT_PULLUP)

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 Circuit Diagram

4.1.1 Complete System Wiring



4.2 Wiring Tables

4.2.1 Node 1 (Arduino UNO) - Complete Wiring

Component	Pin Label	Arduino UNO Pin
MCP2515	VCC	5V
	GND	GND
	CS	D10

Component	Pin Label	Arduino UNO Pin
	SO	D12
	SI	D11
	SCK	D13
	STBY	GND
	CAN H	CAN Bus
	CAN L	CAN Bus
MPU6050	VCC	3.3V
	GND	GND
	SCL	A5
	SDA	A4
	AD0	GND
ACS712	VCC	5V
	GND	GND
	OUT	A0
DS18B20 #1	VDD	5V
	GND	GND
	DATA	D4
DS18B20 #2	VDD	5V
	GND	GND
	DATA	D5

Pull-up resistors: 4.7kΩ between VDD and DATA for each DS18B20

4.2.2 Node 2 (Arduino Nano) - Complete Wiring

Component	Pin Label	Arduino Nano Pin
MCP2515	VCC	5V
	GND	GND
	CS	D10
	SO	D12
	SI	D11
	SCK	D13
	STBY	GND
	CAN H	CAN Bus
	CAN L	CAN Bus
L298N	ENA	D9
	IN1	D8
	IN2	D7
	+5V	5V
	GND	GND
	12V	12V PS
Relay CH1	IN1	D5
	VCC	5V
Relay CH2	IN2	D6
	VCC	5V
Relay Power	DC+	5V

Component	Pin Label	Arduino Nano Pin
	DC-	GND

4.2.3 Node 3 (ESP32) - Complete Wiring

Component	Pin Label	ESP32 GPIO
TJA1050	VCC	5V
	GND	GND
	TXD	GPIO21
	RXD	GPIO22
	CAN H	CAN Bus
	CAN L	CAN Bus
Green LED	Anode	GPIO2
	Cathode	GND (220Ω)
White LED	Anode	GPIO4
	Cathode	GND (220Ω)
Blue LED	Anode	GPIO15
	Cathode	GND (220Ω)
Buzzer	(+)	GPIO16
	(-)	GND
Button	Pin 1	GPIO17
	Pin 2	GND

4.2.4 Node 4 (ESP32) - Complete Wiring

Component	Pin Label	ESP32 GPIO
TJA1050	VCC	5V
	GND	GND
	TXD	GPIO21
	RXD	GPIO22
	CAN H	CAN Bus
	CAN L	CAN Bus
TFT Display	VCC	3.3V
	GND	GND
	CS	GPIO15
	DC	GPIO2
	RST	GPIO4
	MOSI	GPIO23
	SCK	GPIO18
	LED	3.3V

4.2.5 CAN Bus Connections (Between All Nodes)

CAN Bus Connections (Parallel Bus Topology):

Node 1 CAN H ——— Node 2 CAN H ——— Node 3 CAN H ——— Node 4 CAN H

Node 1 CAN L ——— Node 2 CAN L ——— Node 3 CAN L ——— Node 4 CAN L

Node 1 GND ——— Node 2 GND ——— Node 3 GND ——— Node 4 GND

Termination: 120Ω resistor between CAN H and CAN L at Node 1 and Node 4 only.

4.2.6 Termination Jumpers (120Ω)

Node	Jumper Setting	Reason
Node 1 (UNO)	ON	First node on CAN bus
Node 2 (NANO)	OFF	Middle node
Node 3 (ESP32)	OFF	Middle node
Node 4 (ESP32)	ON	Last node on CAN bus

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is used to write, compile, and upload code to all microcontrollers.

Key Features:

- Cross-platform (Windows, Mac, Linux)
- Built-in library manager
- Serial monitor for debugging
- Support for multiple board types

5.2 Libraries Used

5.2.1 CAN.h Library (by Sandeep Mistry)

This library provides CAN bus functionality for ESP32 and Arduino boards.

Key Functions:

Function	Description
CAN.begin(baudrate)	Initialize CAN at specified baud rate
CAN.setPins(rx, tx)	Set CAN RX/TX pins (ESP32 only)
CAN.beginPacket(id)	Start a new CAN message
CAN.write(data)	Write data byte to message
CAN.endPacket()	Send the CAN message
CAN.parsePacket()	Check for incoming message
CAN.read()	Read data byte from received message
CAN.packetId()	Get ID of received packet

CAN Message IDs Used:

Node	CAN ID	Purpose
Node 1	**0x101**	Sensor data
Node 2	**0x200**	Actuator status
Node 3	**0x303**	Heartbeat
Node 4	**0x304**	Heartbeat
Command	**0x401**	Control commands

5.2.2 TFT_eSPI Library (by Bodmer)

This library provides TFT display functionality for ESP32.

Key Functions:

Function	Description
tft.init()	Initialize display
tft.setRotation(rot)	Set screen orientation
tft.fillScreen(color)	Fill entire screen with color
tft.drawString(text, x, y)	Draw text at position
tft.drawRect(x, y, w, h, color)	Draw rectangle outline
tft.fillRect(x, y, w, h, color)	Draw filled rectangle
tft.fillCircle(x, y, r, color)	Draw filled circle
tft.setTextColor(fg, bg)	Set text color

5.2.3 MPU6050 Library (by Electronic Cats)

Function	Description
mpu.initialize()	Initialize sensor
mpu.testConnection()	Check if sensor is connected
mpu.getAcceleration(&ax, &ay, &az)	Get accelerometer data
mpu.getRotation(&gx, &gy, &gz)	Get gyroscope data

5.2.4 OneWire & DallasTemperature Libraries

Function	Description
sensors.begin()	Initialize temperature sensors
sensors.getDeviceCount()	Count connected sensors
sensors.requestTemperatures()	Request temperature reading
sensors.getTempCByIndex(i)	Get temperature in Celsius

5.2.5 MCP2515 Library (by Cory J. Fowler)

Function	Description
mcp2515.reset()	Reset CAN controller
mcp2515.setBitrate(rate, clock)	Set CAN bitrate
mcp2515.setNormalMode()	Set normal operation mode
mcp2515.sendMessage(&frame)	Send CAN message

Function	Description
mcp2515.readMessage(&frame)	Read incoming CAN message

5.3 Data Structures

5.3.1 CAN Message Frame Structure

```
struct can_frame {
  uint32_t can_id;        // 11 or 29-bit identifier
  uint8_t can_dlc;        // Data length code (0-8)
  uint8_t data[8];        // Data bytes
};
```

can_frame structure is used for sending and receiving CAN messages with MCP2515 controller.

5.3.2 Node Status Structures (Node 4)

```
struct Node1Status {
  bool active;                    // Node active flag
  unsigned long lastDataReceived; // Last data timestamp
  bool fault;                     // Fault flag
  float temp1, temp2, current;    // Sensor data
  int16_t accelX;                 // Accelerometer X
};

struct Node2Status {
  bool active;                    // Node active flag
  unsigned long lastDataReceived; // Last data timestamp
  bool fault;                     // Fault flag
  int motorSpeed;                 // Motor speed (0-255)
  bool motorDirection;            // Forward/Reverse
  bool relay1State, relay2State;  // Relay states
};
```

These structures store status data for Node 1 and Node 2 used by Node 3 for fault detection.

5.4 Color Definitions

TFT DISPLAY — RGB565 COLOR REFERENCE

Color	RGB565 Value	Use
Black	0x0000	Background
White	0xFFFF	Text
Red	0xF800	Fault indication
Green	0x07E0	Active/OK status
Yellow	0xFFE0	Warning
Cyan	0x07FF	Header

RGB565 uses 5 bits for Red, 6 bits for Green, 5 bits for Blue. 16-bit color format used by ILI9341 TFT display.

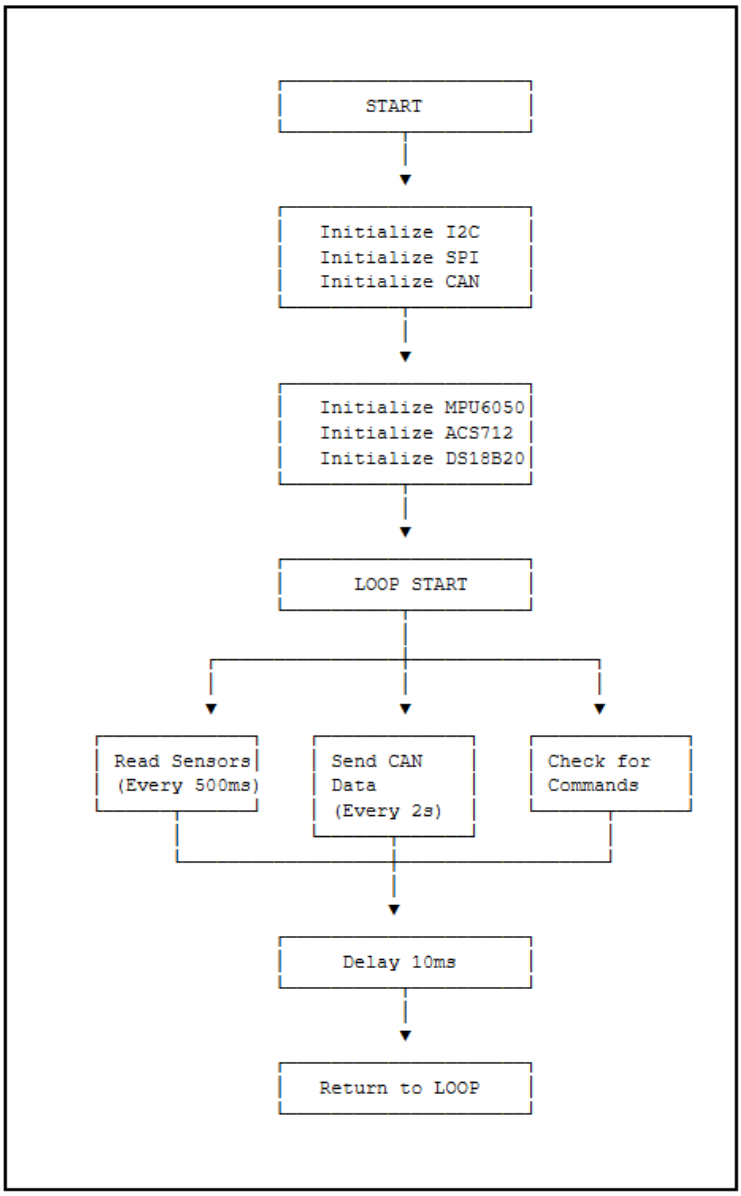
CHAPTER 6

SOFTWARE IMPLEMENTATION

6.1 Flowcharts

6.1.1 Node 1 Flowchart

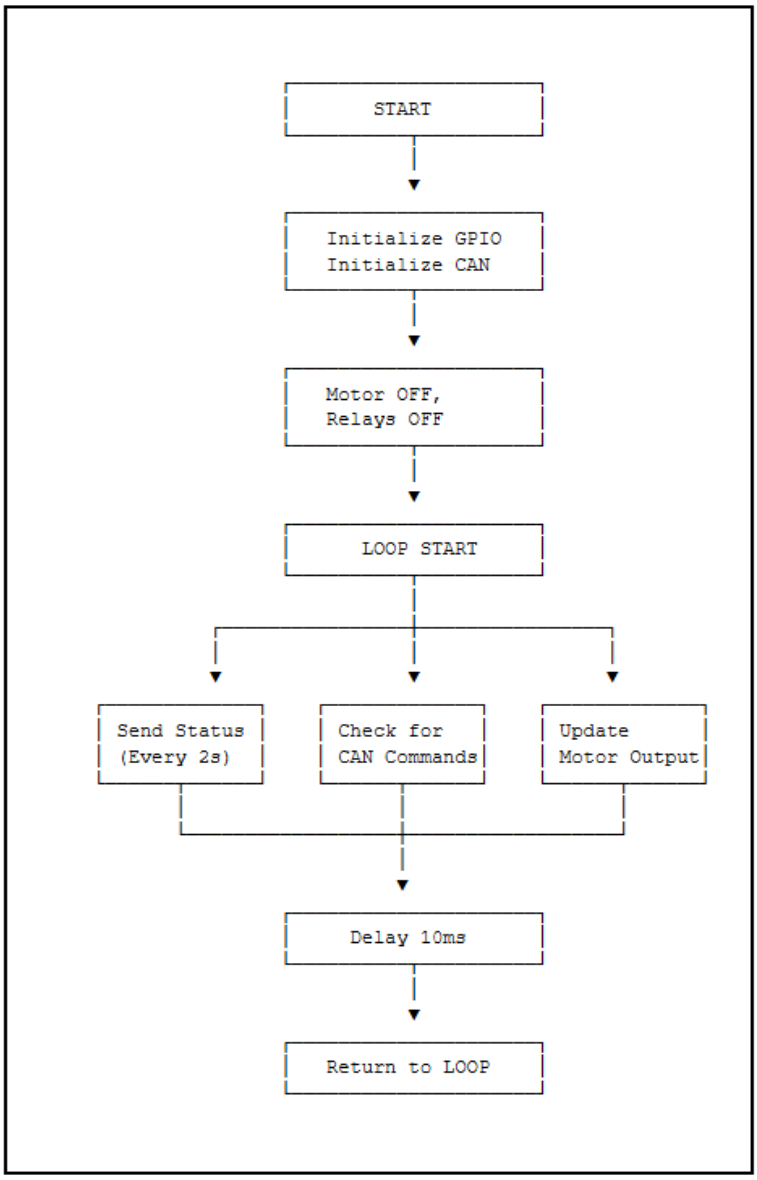
NODE 1 (SENSOR HUB) — FLOWCHART



Node 1 reads temperature, current, and vibration data every 500ms and transmits via CAN every 2 seconds.

6.1.2 Node 2 Flowchart

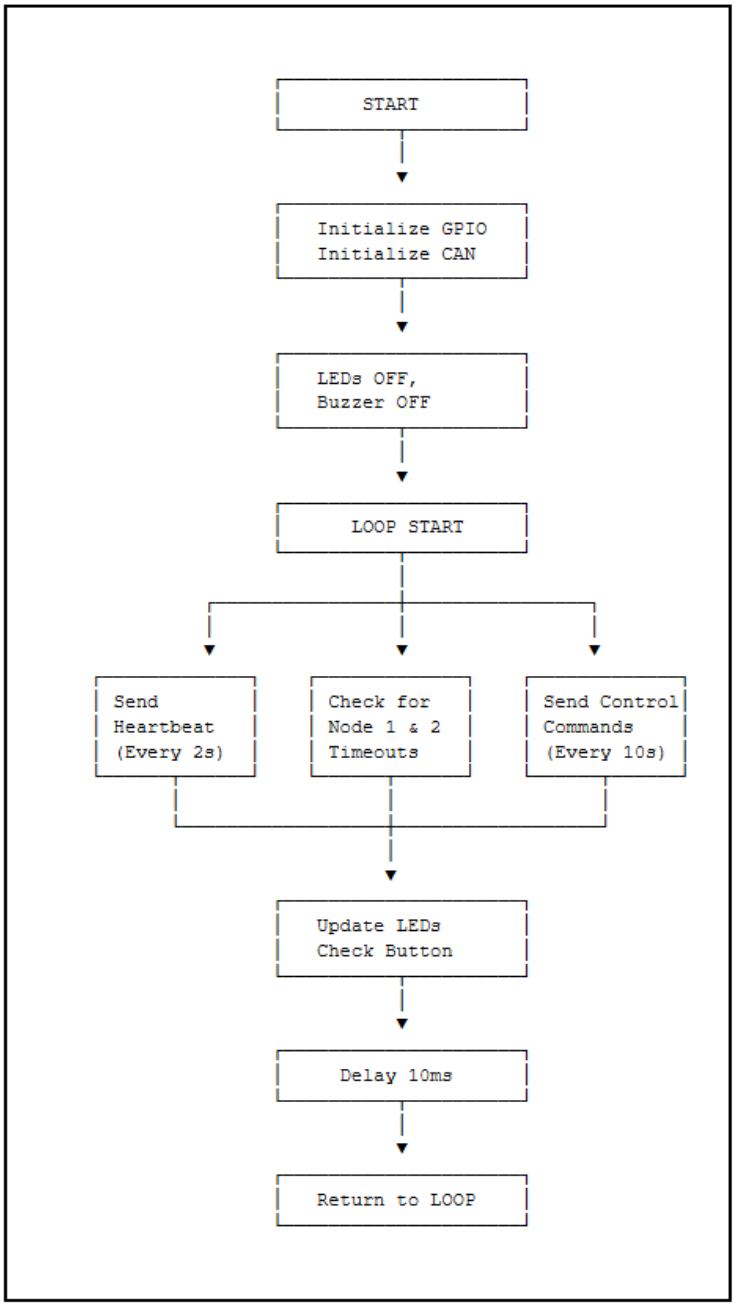
NODE 2 (ACTUATOR CONTROLLER) — FLOWCHART



Node 2 sends actuator status every 2 seconds and listens for CAN commands to control motor and relays.

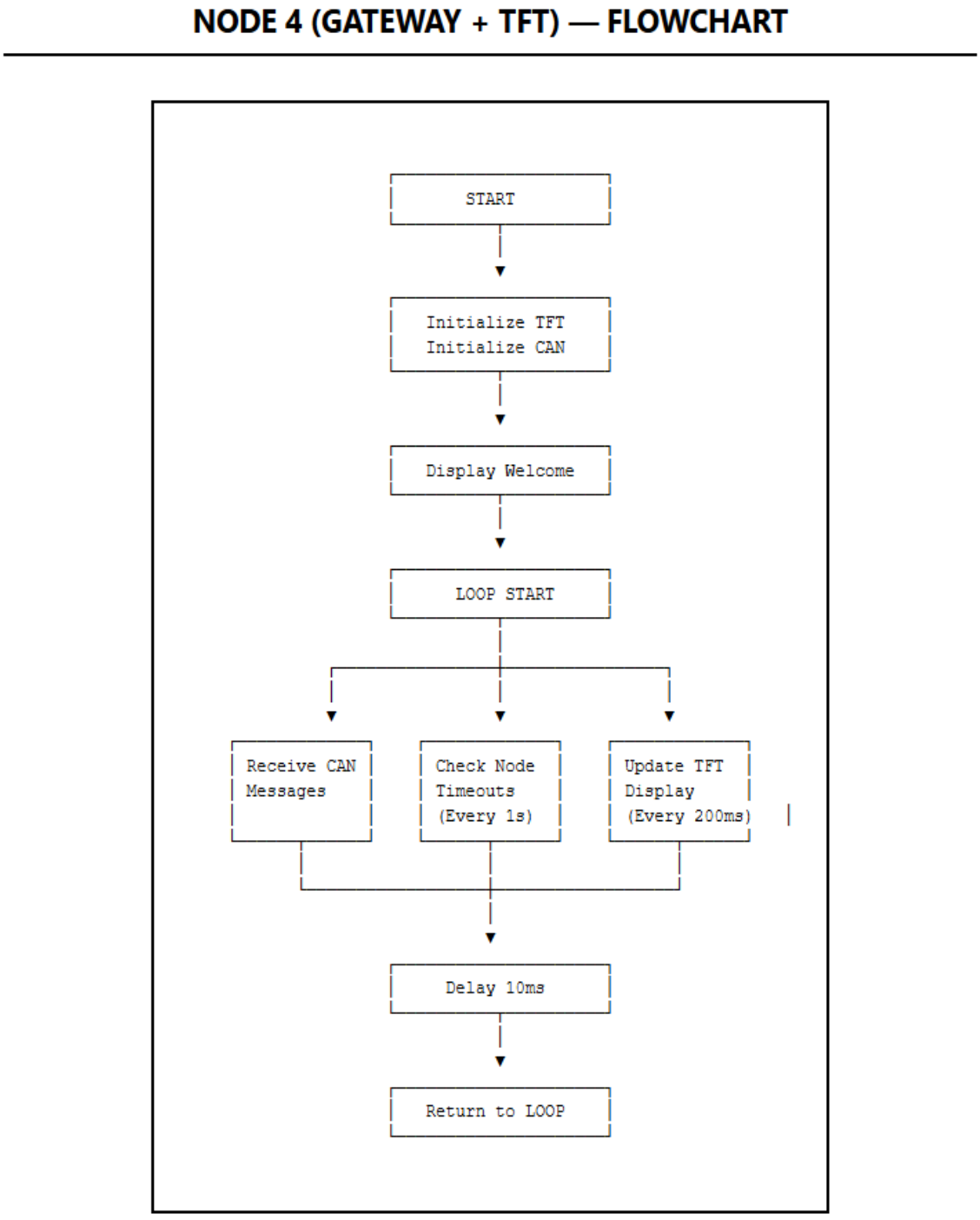
6.1.3 Node 3 Flowchart

NODE 3 (SUPERVISOR) — FLOWCHART



Node 3 monitors Node 1 and Node 2 heartbeats, detects faults (4 sec timeout), and controls LEDs/buzzer.

6.1.4 Node 4 Flowchart



Node 4 receives CAN messages from all nodes, monitors timeouts, and updates TFT display every 200ms.

6.2 Algorithm

6.2.1 Node 1 Algorithm

```
Algorithm: Node1_Main()

1. START

2. Initialize I2C, SPI, and CAN at 250KBPS

3. Initialize MPU6050, ACS712, DS18B20 sensors

4. REPEAT FOREVER:

   4.1 Read all sensors (temperature, current, acceleration)

   4.2 IF time since last send >= 2 seconds THEN
         - Pack sensor data into CAN message (ID: 0x101)
         - Send message
         - Increment sendCount

   4.3 IF CAN message received THEN
         - Parse and print received data

   4.4 Display debug output every 2 seconds

   4.5 Delay 10ms

5. END
```

Node 1 reads sensors every 500ms and transmits CAN data every 2 seconds.

6.2.2 Node 2 Algorithm

```
Algorithm: Node2_Main()

1. START

2. Initialize GPIO pins for motor and relays

3. Initialize CAN at 250KBPS

4. Set initial states: motor OFF, relays OFF

5. REPEAT FOREVER:

   5.1 IF time since last send >= 2 seconds THEN
         - Pack actuator status into CAN message (ID: 0x200)
         - Send message

   5.2 IF CAN message received (ID: 0x401) THEN
         - Extract target, command, value
         - IF target == 2 OR target == 0 THEN
              Execute command (set speed, direction, relay, emergency)

   5.3 Update motor output based on current settings

   5.4 Delay 10ms

6. END
```

Node 2 sends status every 2 seconds and executes commands received from Node 3 (ID: 0x401).

6.2.3 Node 3 Algorithm

```
Algorithm: Node3_Main()

1. START

2. Initialize GPIO for LEDs, buzzer, button

3. Initialize CAN at 250KBPS

4. Set initial states: all LEDs OFF, buzzer OFF

5. REPEAT FOREVER:

   5.1 Send heartbeat every 2 seconds (ID: 0x303)

   5.2 IF time since last node check >= 1 second THEN
         - Check Node 1 timeout (4 seconds)
         - Check Node 2 timeout (4 seconds)
         - IF fault detected THEN sound buzzer and blink blue LED

   5.3 IF time since last command >= 10 seconds THEN
         - Send control command to Node 2 (speed, direction, relays)

   5.4 Update LEDs based on node status

   5.5 IF reset button pressed THEN clear all faults

   5.6 Delay 10ms

6. END
```

Node 3 monitors Node 1 and Node 2, sends control commands, and provides visual/audible fault indication.

6.2.4 Node 4 Algorithm

```
Algorithm: Node4_Main()

1. START

2. Initialize TFT display (320x240, rotation 1)

3. Initialize CAN at 250KBPS

4. Display "Initializing..." on TFT

5. REPEAT FOREVER:

   5.1 Receive CAN messages:
         - IF ID == 0x101 THEN update Node 1 data
         - IF ID == 0x200 THEN update Node 2 data
         - IF ID == 0x303 THEN update Node 3 heartbeat

   5.2 Check timeouts every 1 second:
         - IF no data from Node 1 for 4 seconds THEN mark fault
         - IF no data from Node 2 for 4 seconds THEN mark fault
         - IF no heartbeat from Node 3 for 4 seconds THEN mark fault

   5.3 Update TFT display every 200ms:
         - Show Node 1 sensor data
         - Show Node 2 actuator data
         - Show system status (NORMAL/FAULT/WAITING)

   5.4 Send heartbeat every 2 seconds (ID: 0x304)

   5.5 Delay 10ms

6. END
```

Node 4 aggregates all CAN data, detects faults, and provides real-time visualization on TFT display.

CHAPTER 7

EXPERIMENTAL RESULTS

7.1 Test Setup

The system was tested with all 4 nodes powered and connected via CAN bus. Each node's serial monitor was observed simultaneously to verify communication.

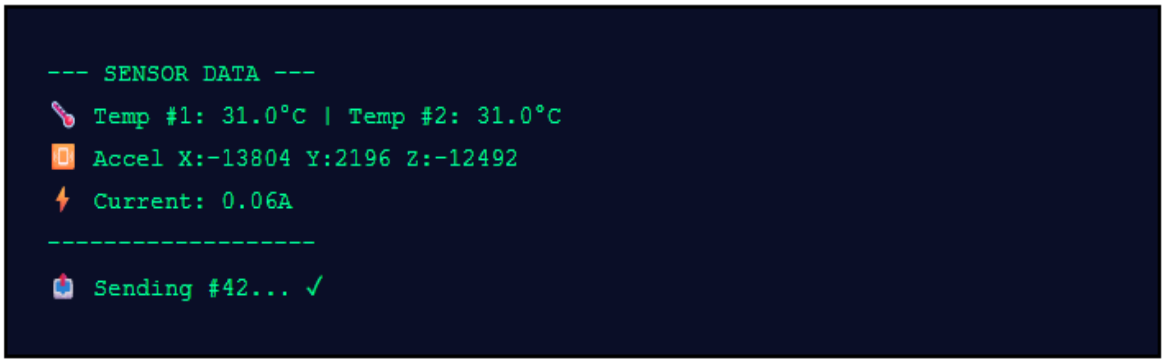
Test Conditions:

- CAN bitrate: 250KBPS
- Power supply: 12V 2A adapter with LM2596 buck converter (5V)
- Ambient temperature: 30-32°C
- Test duration: Continuous operation for 1 hour

7.2 Node 1 Test Results

Sensor	Reading
DS18B20 #1	31.0°C
DS18B20 #2	31.0°C
ACS712 Current	0.06A
MPU6050 Accel X	-13804
CAN Transmission	Every 2 seconds

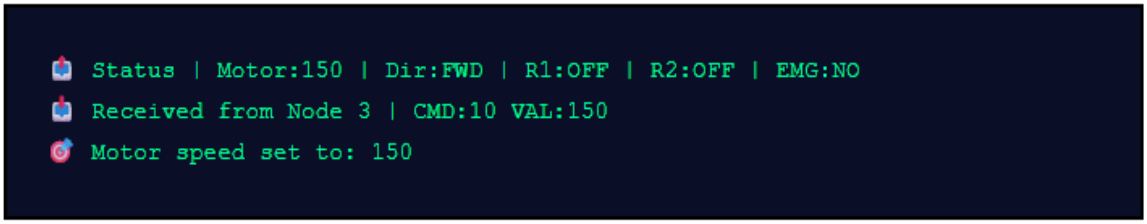
Sample Serial Output (Node 1):



7.3 Node 2 Test Results

Actuator	Test
Motor Speed	0-255
Motor Direction	Forward/Reverse
Relay 1	ON/OFF
Relay 2	ON/OFF
Emergency Stop	Active/Inactive
CAN Reception	ID: 0x401

Sample Serial Output (Node 2):

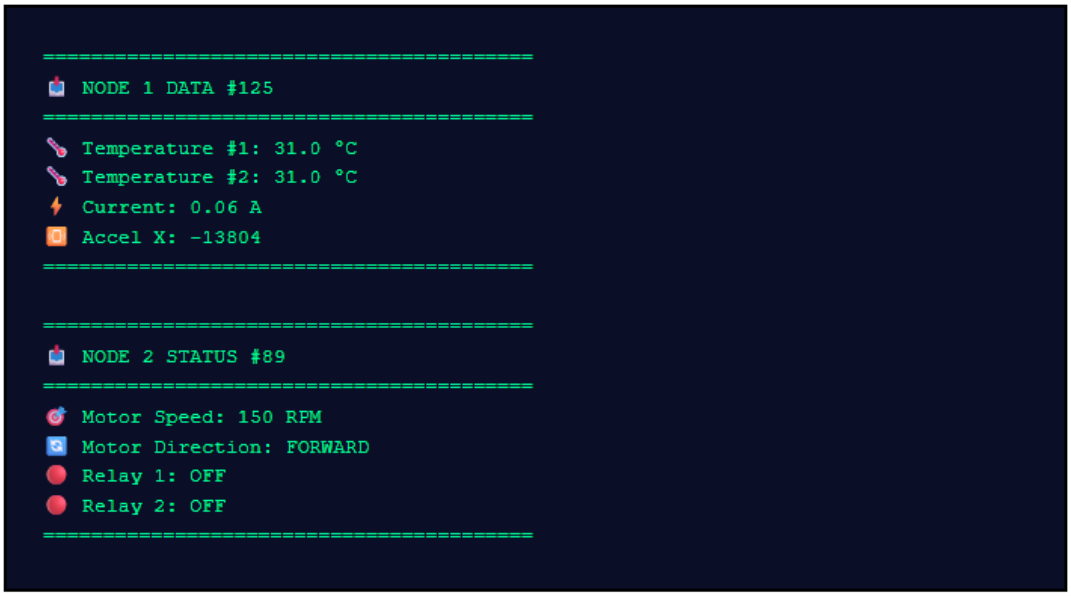


7.4 Node 3 Test Results

Function	Test
Node 1 Monitoring	Data reception
Node 2 Monitoring	Data reception
Green LED	Node 1 active
White LED	Node 2 active
Blue LED	Fault detection

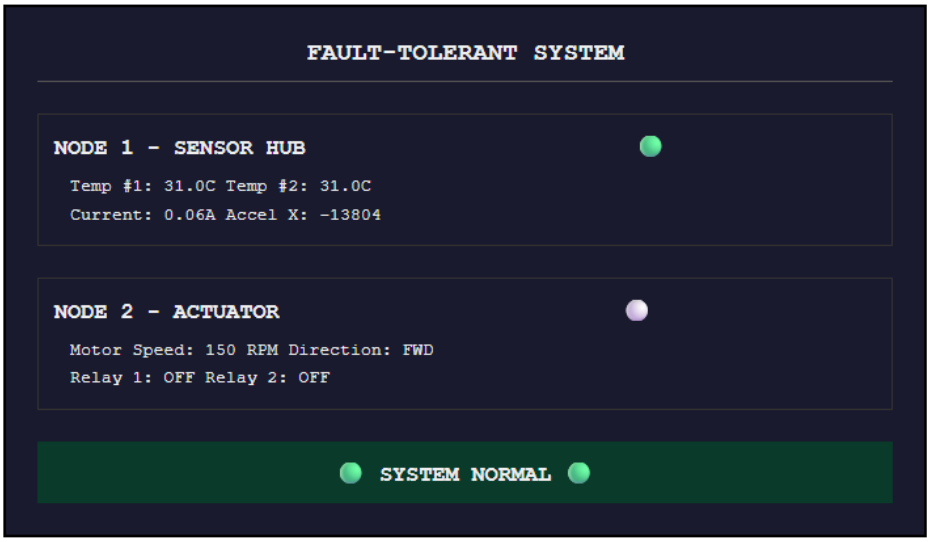
Function	Test
Buzzer	Fault detection
Reset Button	Fault clear
CAN Commands	Sent every 10s

Sample Serial Output (Node 3):

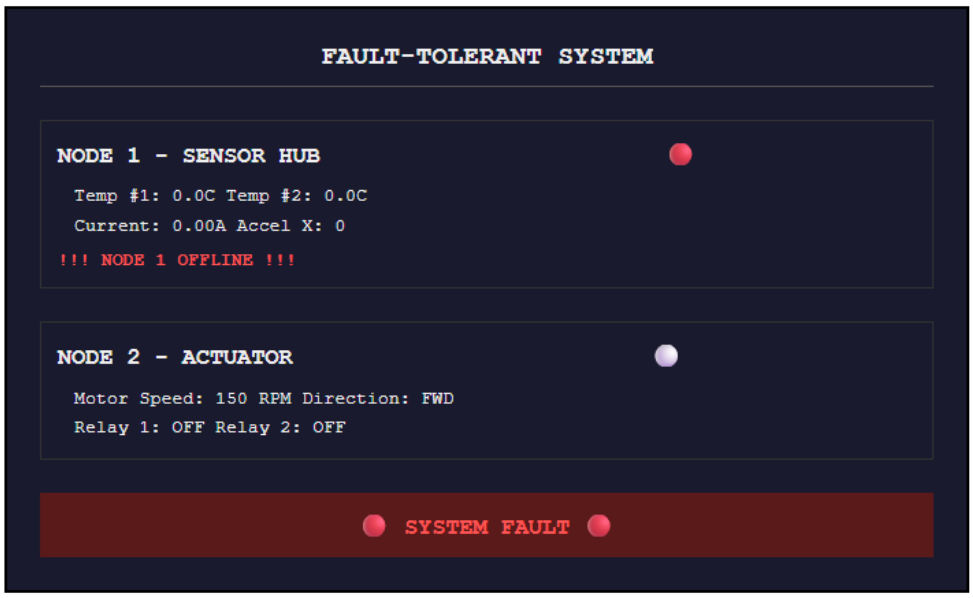


7.5 Node 4 TFT Display Results

7.5.1 Normal Operation Display



7.5.2 Fault Condition Display (Node 1 Disconnected)



7.6 Fault Tolerance Test Results

Test Scenario	Expected Response	Actual Response
Disconnect Node 1	Fault detected in 4 seconds	Blue LED blinks, buzzer sounds, TFT shows SYSTEM FAULT
Reconnect Node 1	System recovers	Blue LED OFF, buzzer OFF, TFT shows SYSTEM NORMAL
Disconnect Node 2	Fault detected in 4 seconds	Blue LED blinks, buzzer sounds, TFT shows SYSTEM FAULT
Press Reset Button	All faults cleared	Blue LED OFF, buzzer OFF, TFT shows SYSTEM NORMAL
Disconnect Node 3	TFT shows SYSTEM FAULT	SYSTEM FAULT displayed on TFT

7.7 Power Consumption

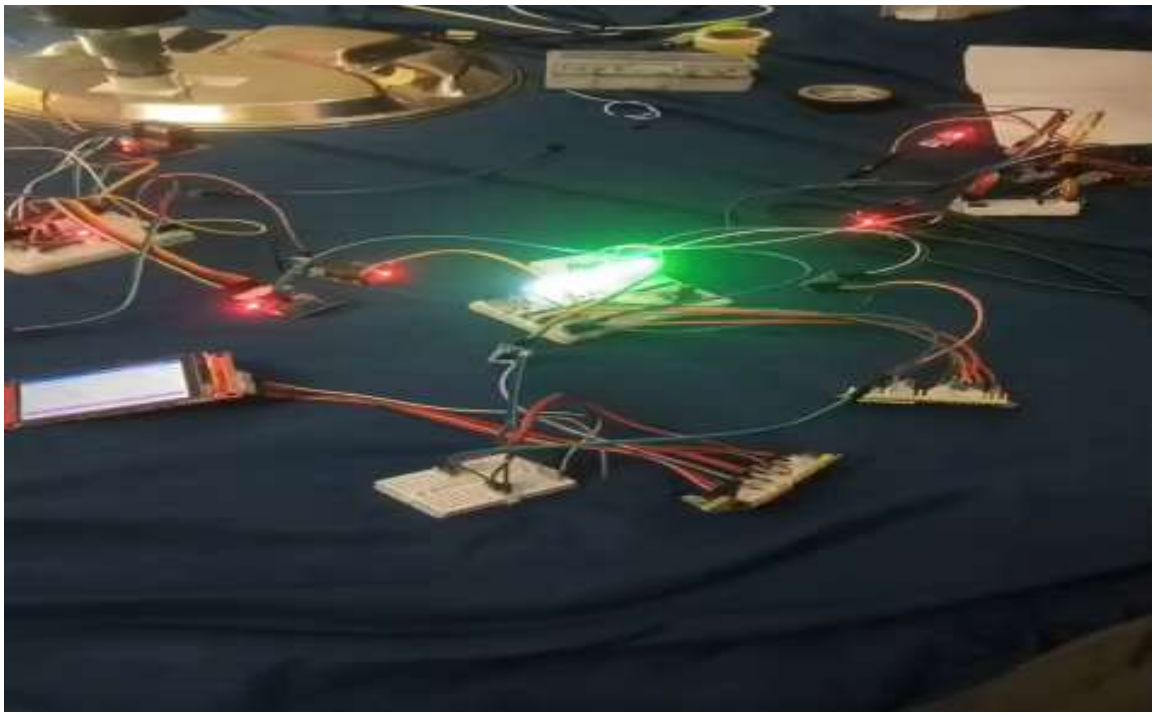
Node	Voltage	Current	Power
Node 1 (UNO + Sensors)	5V	~150mA	0.75W

Node	Voltage	Current	Power
Node 2 (Nano + Motor + Relays)	5V (logic), 12V (motor)	~200mA + motor current	~1W + motor
Node 3 (ESP32)	5V	~100mA	0.5W
Node 4 (ESP32 + TFT)	5V	~150mA	0.75W
Total (excluding motor)	**-**	**~600mA**	**~3W**

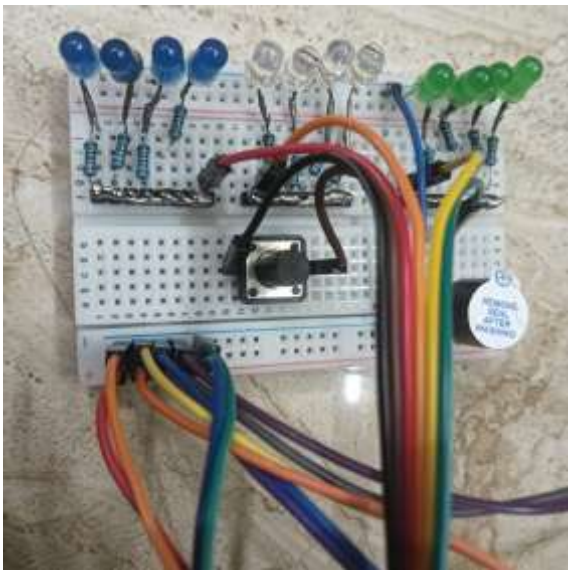
7.8 Performance Metrics

Metric	Value
CAN Bitrate	250 KBPS
CAN Bus Length	~1m (test setup)
Node 1 Data Rate	Every 2 seconds
Node 2 Status Rate	Every 2 seconds
Node 3 Command Rate	Every 10 seconds
TFT Update Rate	Every 200ms
Fault Detection Time	4 seconds
Power Supply	12V 2A adapter

7.9 Hardware Setup Photo



Test Results:



CHAPTER 8

APPLICATIONS, ADVANTAGES AND DISADVANTAGES

8.1 Applications

8.1.1 Industrial Automation

Application	Description
Motor Monitoring	Real-time monitoring of motor temperature, current, and vibration
Predictive Maintenance	Detect early signs of motor failure through vibration analysis
Factory Automation	Distributed control of conveyor belts, robotic arms
Process Control	Temperature and current monitoring in manufacturing processes

8.1.2 Automotive Systems

Application	Description
Engine Control Units	Multiple ECU communication via CAN bus
Electric Vehicle	Battery monitoring, motor control
Vehicle Diagnostics	Real-time fault detection and reporting

8.1.3 Building Management

Application	Description
HVAC Systems	Temperature monitoring and control

Application	Description
Lighting Control	Relay-based lighting automation
Fire Alarm Systems	Distributed sensor network with fault detection

8.1.4 Education and Research

Application	Description
Embedded Systems Lab	Teaching CAN protocol and distributed systems
Research Projects	Testing fault-tolerant algorithms
Student Projects	Learning multi-node communication

8.2 Advantages

Advantage	Description
Fault Tolerance	System continues to function even if one node fails
Distributed Architecture	No single point of failure
Real-time Monitoring	All sensor and actuator data displayed on TFT
Scalability	Easy to add more nodes to the CAN bus
Low Wiring Cost	Only 3 wires needed for all nodes (CAN H, CAN L, GND)
Industrial Standard	CAN bus is widely used in automotive and industrial applications
Visual Feedback	LEDs, buzzer, and TFT display provide clear system status

Advantage	Description
Redundant Sensing	Two DS18B20 sensors provide backup temperature monitoring
Manual Recovery	Reset button allows quick fault recovery
Cross-platform	Different microcontrollers (UNO, Nano, ESP32) work together

8.3 Disadvantages

Disadvantage	Description	Mitigation
Manual Reset Required	System does not auto-recover from faults	Can be enhanced with software auto-recovery
No Data Logging	Fault events are not logged	Can add SD card module
No Remote Monitoring	Cannot monitor system remotely	Can add Wi-Fi to ESP32 nodes
Limited CAN Speed	250KBPS (can go up to 1MBPS)	Increase bitrate for shorter cables
Power Supply Dependency	Single power supply for all nodes	Add redundant power supply
No Automatic Failover	Failed node not automatically replaced	Can add backup nodes
Limited Node Distance	CAN bus length limited at 250KBPS (~250m)	Use repeaters for longer distance

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The **Fault-Tolerant Distributed Embedded Control System** has been successfully designed, implemented, and tested. The project achieved all its objectives:

Objectives Achieved:

Objective	Status
4-node distributed system using CAN bus	ACHIEVED
Real-time sensor monitoring (temperature, current, vibration)	ACHIEVED
Actuator control (motor speed, direction, relays)	ACHIEVED
Fault detection using heartbeat monitoring	ACHIEVED
Visual and audible fault indication	ACHIEVED
TFT display showing all system data	ACHIEVED
Manual fault recovery via reset button	ACHIEVED

Key Learnings:

1. **CAN Bus Protocol** - Understanding of multi-master communication, arbitration, and message framing
2. **Distributed Systems** - Coordination between independent nodes
3. **Fault Tolerance** - Heartbeat monitoring and timeout detection
4. **Sensor Integration** - I2C (MPU6050), 1-Wire (DS18B20), Analog (ACS712)
5. **Actuator Control** - PWM motor control, relay switching
6. **TFT Display Programming** - GUI development for embedded systems
7. **Multi-platform Development** - Arduino UNO, Nano, and ESP32 working together

The system is **complete, functional, and ready for demonstration**. It demonstrates professional-grade fault-tolerant distributed control suitable for industrial applications.

9.2 Future Scope

Enhancement	Description	Priority
Automatic Failover	System automatically re-routes control when node fails	High
Wi-Fi/Bluetooth Monitoring	Remote monitoring via smartphone app	High
Data Logging	SD card storage of sensor data and fault events	Medium
Cloud Dashboard	Real-time data visualization on cloud platform	Medium
Predictive Maintenance	Machine learning for fault prediction	Medium
Additional Nodes	Add more sensor/actuator nodes	Low
Touch Interface	Add touch control to TFT display	Low
Battery Backup	Redundant power supply for true fault tolerance	Low
CANopen Protocol	Implement higher-level CAN protocol	Low
Real-time Clock	Timestamp all data and faults	Low

Recommended Next Steps:

1. **Add Wi-Fi to ESP32 nodes** for remote monitoring via Blynk or Arduino IoT Cloud
2. **Add SD card module** to Node 4 for data logging
3. **Implement auto-recovery** using watchdog timers
4. **Create mobile app** for remote system monitoring

REFERENCES

1. **Bosch CAN Specification Version 2.0**, Robert Bosch GmbH, 1991.
2. **MCP2515 Data Sheet**, Microchip Technology Inc., 2005.
3. **TJA1050 Data Sheet**, NXP Semiconductors, 2015.
4. **MPU6050 Data Sheet**, InvenSense Inc., 2013.
5. **ACS712 Data Sheet**, Allegro MicroSystems, 2017.
6. **DS18B20 Data Sheet**, Maxim Integrated, 2019.
7. **L298N Data Sheet**, STMicroelectronics, 2000.
8. **ESP32 Technical Reference Manual**, Espressif Systems, 2021.
9. **Arduino UNO R3 Product Reference**, Arduino.cc
10. **TFT_eSPI Library Documentation**, Bodmer, GitHub.
11. **CAN.h Library Documentation**, Sandeep Mistry, GitHub.
12. **MPU6050 Library Documentation**, Electronic Cats, GitHub.